

Programação Funcional

Generalização em Haskell

Vladimir Oliveira Di Iorio
DPI - UFV

Funções como argumentos

- Assim como números, tuplas e listas, também **funções** podem ser argumentos para outras funções.
- Funções que recebem outras funções como argumentos são chamadas ***funções de ordem mais alta.***

Motivação

- Função para retornar uma lista com os elementos "dobrados":

```
1 double :: [Int] -> [Int]
2 double [] = []
3 double (a:x) = (2*a) : double x
```

- Função para retornar uma lista com os elementos "triplicados":

```
1 treble :: [Int] -> [Int]
2 treble [] = []
3 treble (a:x) = (3*a) : treble x
```

Motivação

```
1 double :: [Int] -> [Int]
2 double [] = []
3 double (a:x) = (2*a) : double x
```

```
1 treble :: [Int] -> [Int]
2 treble [] = []
3 treble (a:x) = (3*a) : treble x
```

- O padrão de ambas as definições é o mesmo.
- A única coisa que muda é o modo como os elementos da lista são transformados.

Generalizando

- Uma função genérica para transformar listas de inteiros deveria então ter **dois** argumentos:
 - uma função que especifica a transformação a ser realizada em cada elemento da lista;
 - a lista a ser transformada.
- Exemplos de funções de transformação para elementos (inteiros):

```
1 times2, times3 :: Int -> Int
2 times2 n = 2*n
3 times3 n = 3*n
```

Generalizando

- Função genérica para transformar listas de inteiros:

```
1 mapInt f [] = []  
2 mapInt f (a:x) = (f a) : mapInt f x
```

- A função `mapInt` recebe como argumentos:
 - uma função `f` que especifica a transformação a ser realizada em cada elemento da lista;
 - a lista a ser transformada.

Generalizando - outra solução

- Função genérica para transformar listas de inteiros:

```
1 mapInt f [] = []  
2 mapInt f (a:x) = (f a) : mapInt f x
```

- Outra solução - usando *list comprehensions*:

```
1 mapInt f l = [ (f a) | a <- l]
```

Exemplo

```
1 mapInt f [] = []  
2 mapInt f (a:x) = (f a) : mapInt f x  
3  
4 times2 n = 2*n
```

```
MapInt times2 [1,4]  
= (times2 1) : mapInt times2 [4]  
= 2 : mapInt times2 [4]  
= 2 : ((times2 4) : mapInt times2 [])  
= 2 : (8 : mapInt times2 [])  
= 2 : (8 : [])  
= [2,8]
```

Exemplo

```
1 mapInt f [] = []  
2 mapInt f (a:x) = (f a) : mapInt f x  
3  
4 times3 n = 3*n
```

```
MapInt times3 [1,4]  
= (times3 1) : mapInt times3 [4]  
= 3 : mapInt times3 [4]  
= 3 : ((times3 4) : mapInt times3 [])  
= 3 : (12 : mapInt times3 [])  
= 3 : (12 : [])  
= [3,12]
```

Redefinindo double e treble

```
1 mapInt f [] = []
2 mapInt f (a:x) = (f a) : mapInt f x
3
4 times2, times3 :: Int -> Int
5 times2 n = 2*n
6 times3 n = 3*n
7
8 double l = mapInt times2 l
9 treble l = mapInt times3 l
```

```
Main> double [1,4]
[2,8]
Main> treble [1,4]
[3,12]
```

Definindo tipos

```
1 mapInt :: t1??? -> t2??? -> t3???  
2 mapInt f [] = []  
3 mapInt f (a:x) = (f a) : mapInt f x
```

- Qual o tipo de `mapInt` ?
 - o primeiro argumento é uma função que transforma um inteiro em outro inteiro;
 - o segundo argumento é uma lista de inteiros;
 - retorna outra lista de inteiros.

Definindo tipos

```
1 mapInt :: (Int->Int) -> [Int] -> [Int]
2 mapInt f [] = []
3 mapInt f (a:x) = (f a) : mapInt f x
```

- Qual o tipo de mapInt ?
 - o primeiro argumento é uma função que transforma um inteiro em outro inteiro;
 - o segundo argumento é uma lista de inteiros;
 - retorna outra lista de inteiros.

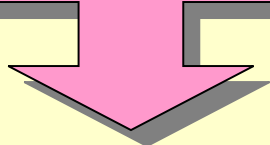
Vantagens

- Ao usar funções de ordem mais alta, consegue-se mais clareza e simplicidade nas definições.
- Fica mais fácil proceder modificações. Por exemplo, se for necessário modificar apenas a função de transformação de elementos.
- Permite **reutilização** de definições. A função `MapInt` pode ser usada em muitas outras situações.

Exemplo 1

```
1 sales :: Int -> Int
2 sales x
3   | x == 0      = 12
4   | x == 1      = 20
5   | x == 2      = 18
6   | x == 3      = 25
7   | otherwise   = 0
```

Função totalSales usa
uma definição
específica para sales.



```
8 totalSales :: Int -> Int
9 totalSales n
10  | n == 0      = sales 0
11  | otherwise   = totalSales (n-1) + sales n
```

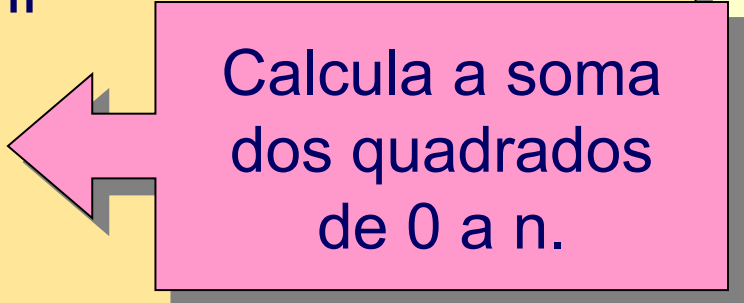
Exemplo 1 - generalizando

```
8 total :: (Int -> Int) -> Int -> Int
9 total f n
10   | n==0           = f 0
11   | otherwise      = total f (n-1) + f n
12
13 totalSales n = total sales n
```

- A função `total` pode ser usada em muitas outras situações...

Exemplo 1 - generalizando

```
8 total :: (Int -> Int) -> Int -> Int
9 total f n
10   | n==0           = f 0
11   | otherwise      = total f (n-1) + f n
12
13 totalSales n = total sales n
14
15 sumSquares :: Int -> Int
16 sumSquares n = total sq n
17
18 sq :: Int -> Int
19 sq x = x*x
```



Calcula a soma
dos quadrados
de 0 a n.

Exemplo 2

- Soma dos elementos de uma lista de inteiros:
$$e1 + e2 + \dots + e_k$$
- Máximo elemento de uma lista de inteiros:
$$e1 \text{ 'maxi' } e2 \text{ 'maxi' } \dots \text{ 'maxi' } e_k$$
- Tipos das funções aplicadas a **pares** de elementos:
$$(+) , \text{ 'maxi' } :: \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$$

Exemplo 2 - generalizando

```
1 foldInt :: (Int -> Int -> Int) -> [Int] -> Int
2
3 foldInt f [a]      = a
4 foldInt f (a:b:x)  = f a (foldInt f (b:x))
5
6
7 sumList l = foldInt (+) 1
8 maxList l = foldInt maxi 1
```

Exemplo 3

- Filtrar algarismos de um texto:

```
digits "29 February 1996"  
= "291996"
```

- Filtrar letras de um texto:

```
letters "29 February 1996"  
= "February"
```

- Tipos das funções aplicadas a cada elemento:

```
isDigit, isLetter :: Char -> Bool
```

Exemplo 3 - generalizando

```
1 filterString :: (Char -> Bool) -> [Char] -> [Char]
2
3 filterString p [] = []
4 filterString p (a:x)
5   | p a           = a : filterString p x
6   | otherwise     = filterString p x
7
8 isDigit, isLetter :: Char -> Bool
9 isDigit ch = ('0'<=ch && ch<='9')
10 isLetter ch = ('a'<=ch && ch <='z') ||
11              ('A'<=ch && ch <='Z')
12
13 digits st = filterString isDigit st
14 letters st = filterString isLetter st
```

Exemplo 3 - outra solução

```
1 filterString :: (Char -> Bool) -> [Char] -> [Char]
2
3 filterString p x = [ a | a<-x , p a ]
4
5 isDigit, isLetter :: Char -> Bool
6 isDigit ch = ('0'<=ch && ch<='9')
7 isLetter ch = ('a'<=ch && ch <='z') ||
8               ('A'<=ch && ch <='Z')
9
10 digits st = filterString isDigit st
11 letters st = filterString isLetter st
```

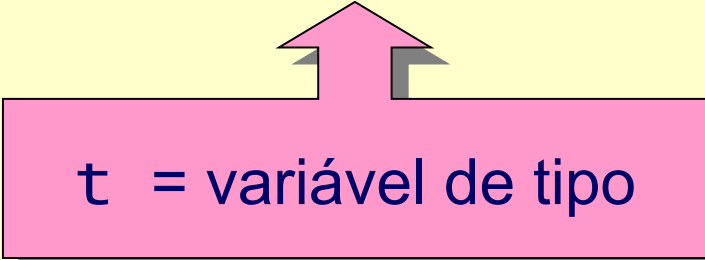
Polimorfismo

- Função que calcula o comprimento de uma lista:

```
1 . length [] = 0
2 . length (a:x) = 1 + length x
```

- Qual o tipo de length ?

`length :: [t] -> Int`



`t = variável de tipo`

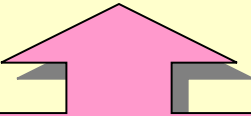
Outros exemplos de polimorfismo

- Função que calcula o reverso de uma lista:

```
1 rev [] = []  
2 rev (a:x) = rev x ++ [a]
```

- Qual o tipo de rev ?

rev :: [t] -> [t]



Importante: retorna lista do mesmo tipo
que o argumento de entrada

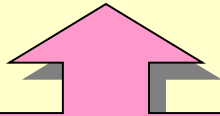
Outros exemplos de polimorfismo

- Função para construir lista de pares:

```
1 zip (a:x) (b:y) = (a,b) : zip x y
2 zip _ _ = []
```

- Qual o tipo de zip ?

`zip :: [t] -> [u] -> [ (t,u) ]`



Importante: os tipos `t` e `u`
não estão relacionados!

Vantagens

- Ao usar polimorfismo, as definições ficam mais genéricas.
- As definições podem ser reutilizadas em muitas situações diferentes.
- Por exemplo, a função `length` calcula o comprimento de listas de qualquer tipo.
- Sem polimorfismo, seria necessário criar uma versão de `length` para cada tipo de lista a ser utilizada.